Materia: Programación IV

UNIDAD 6 · RELACIONES Y ESTADO

APUNTE TEÓRICO

Formularios con Custom Hooks en React + TypeScript

Revisor de la unidad: Prof. Juan Ignacio Sarmiento

En aplicaciones modernas desarrolladas con React, la gestión de formularios es uno de los problemas más frecuentes y repetitivos. Cuando no se abstrae correctamente, suele generar:

- Código duplicado.
- Múltiples funciones `onChange`.
- Estados dispersos.
- Baja escalabilidad.
- Dificultad de mantenimiento.

Para resolver esto, se implementa un **Custom Hook genérico**, que encapsula la lógica del formulario y la hace reutilizable en toda la aplicación.

Abstracción del Estado del Formulario

El enfoque tradicional en React consiste en:

- Crear un `useState` por cada campo.
- Crear un `onChange` por cada input.
- Gestionar manualmente cada actualización.

Este patrón no escala bien cuando el formulario crece.

El enfoque profesional consiste en:

- Centralizar todos los campos en un único objeto.
- Usar una función dinámica de actualización.
- Preservar tipado estricto con TypeScript.
- Mantener inmutabilidad del estado.

El custom hook implementado cumple exactamente esa función.

Diseño del Hook Genérico

El hook se define utilizando genéricos de TypeScript:

```
<T extends Record<string, unknown>>
```

Esto significa que:

- El hook puede trabajar con cualquier estructura de formulario.
- No está acoplado a un modelo específico.
- Mantiene type safety.
- Se adapta dinámicamente al objeto inicial.

Desde una perspectiva arquitectónica, esto convierte al hook en una herramienta reutilizable de dominio transversal.

Implementación del hook (`hooks/useForm.ts`):

```
export const useForm = <T extends Record<string, unknown>>(initialState: T) => {
  const [formState, setFormState] = useState(initialState);

  const handleChange = (
    e: ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>
  ) => {
    const { name, value } = e.target;
    setFormState((prevState) => ({
      ...prevState,
      [name]: value,
    }));
  };

  const resetForm = () => {
    setFormState(initialState);
  };

  return {
    ...formState,
    formState,
    handleChange,
    resetForm,
  };
};
```

`handleChange` es la función única del punto anterior: lee `name` y `value` del evento y actualiza esa sola propiedad del objeto de estado, preservando el resto con el spread operator. `resetForm` vuelve el formulario a los valores iniciales recibidos por parámetro.

Centralización del Estado

En lugar de múltiples estados independientes, se utiliza un único objeto:

Conceptualmente:

```
formValues = {
  nombre: '',
  descripcion: ''
}
```

Ventajas profesionales:

- Escalabilidad inmediata.
- Claridad estructural.
- Reducción de errores.
- Facilidad para serializar datos.
- Integración simple con APIs REST.

Este enfoque respeta el principio de **estado mínimo necesario**.

Actualización Dinámica mediante name

El patrón clave del hook es que cada input tiene un atributo `name`.

Ese atributo permite que una única función pueda actualizar cualquier campo.

Conceptualmente el proceso es:

1. El usuario modifica un input.
2. El evento contiene el `name`.
3. Se actualiza dinámicamente la propiedad correspondiente.
4. Se preservan los demás valores mediante spread operator.

Este patrón:

- Elimina múltiples handlers.
- Reduce código repetitivo.
- Mantiene inmutabilidad.
- Facilita mantenimiento futuro.

Desde una mirada profesional, esta técnica es estándar en proyectos de producción.

Extensión para Casos Avanzados

La versión extendida del hook agrega una función que permite modificar el estado desde el exterior.

Esto habilita:

- Precargar datos para edición.
- Reemplazar todo el formulario.
- Integrar datos provenientes de API.
- Sincronizar con estado global.

Este detalle es importante en arquitecturas reales, donde los formularios no siempre comienzan vacíos.

Especialización del Hook en Dominio de Categorías

En lugar de usar directamente el hook genérico en toda la aplicación, se crea una especialización:

```
useCategoriasForm()
```

Este patrón tiene ventajas arquitectónicas:

- Se desacopla la lógica genérica del dominio específico.
- Se mejora la semántica del código.
- Se facilita mantenimiento.
- Se prepara la aplicación para crecimiento.

Esto aplica el principio de separación entre infraestructura y dominio.

Aplicación en un Modal Controlado

El componente `CategoriaModal` utiliza el hook para gestionar su propio estado interno.

Desde una perspectiva de arquitectura:

- El modal no conoce cómo se guardan las categorías.
- Solo recibe callbacks.
- No tiene lógica de negocio.
- Solo gestiona presentación y captura de datos.

Se respetan principios fundamentales:

- Separación de responsabilidades.

- Componentes controlados.
- Flujo de datos unidireccional.
- Inmutabilidad del estado.

Cuando el usuario confirma:

1. Se ejecuta `onSubmit`.
2. Se reinicia el formulario.
3. Se delega la actualización al componente padre.

Esto mantiene el control de la lógica en un nivel superior.

Integración en el Componente Principal

El componente `App` cumple el rol de orquestador.

Responsabilidades:

- Mantener el estado global de categorías.
- Abrir y cerrar el modal.
- Agregar nuevas categorías.
- Generar identificadores dinámicos.
- Mantener coherencia temporal (`created_at`, `updated_at`).

Se observa aquí un patrón profesional:

- El estado vive en el nivel más alto necesario.
- Los componentes hijos reciben datos y callbacks.
- Las actualizaciones se realizan de forma inmutable.
- Se utiliza función actualizadora para evitar estados inconsistentes.

Este modelo es escalable hacia:

- Persistencia en base de datos.
- Integración con API REST.
- Reemplazo por Zustand o Redux sin modificar la UI.

Bibliografía

1. Banks, A., & Porcello, E. (2020). Learning React (2.ª ed.). O'Reilly Media.
2. Cherny, B. (2019). Programming TypeScript. O'Reilly Media.
3. Microsoft. (s. f.). TypeScript handbook. Recuperado el 2 de octubre de 2026, de https://www.typescriptlang.org/docs/handbook
4. React. (s. f.). React documentation. Recuperado el 2 de octubre de 2026, de https://react.dev